Estensione di un domain-specific language per le misure Internet

Giovanni Cisternini

Indice

1 Introduzione **2**

2 Architettura **2**
2.1 RIPE Atlas 2
2.1.1 Probe 2
2.1.2 Measurement 3
2.1.3 Result 4
2.2 DSL con Groovy 4
2.2.1 Closures 4
2.2.2 Maps 6
2.2.3 Command Chain 6

3 Implementazione DSL **7**
3.1 Builder 8
3.2 Parametri e operazioni implementati 9
3.3 Metodo Traceroute 10
3.3.1 Filtri 19
3.3.2 API per ASN e coordinate 20

4 Modifiche **21**
4.1 Modifica dell'intervallo di tempo per i measurement 21

5 Esempio di Utilizzo **22**

6 Conclusioni **26**

1 Introduzione

Possiamo definire un Domain-Specific Language (abbr. DSL) come un linguaggio di programmazione sviluppato appositamente per un dominio di applicazione, sul quale poi si estendono le caratteristiche del linguaggio[Dea10a]. Utilizzando un DSL è possibile realizzare programmi ad alto livello di astrazione, maggiormente specializzati, più sintetici e leggibili rispetto a programmi implementati in linguaggi classici. La tesi prende in esame l'estensione di un DSL già implementato, il quale permette di analizzare le misurazioni delle prestazioni di Internet attraverso l'infrastruttura fornita da RIPE Atlas. Quest'ultima è costituita da una rete di dispositivi (probes), i quali effettuano misurazioni (measurements) di diverso tipo sulla connettività e raggiungibilità di Internet. Nello specifico, il DSL prevede già la possibilità di ottenere misurazioni di tipo Ping, si vogliono quindi estendere le tipologie di misure analizzabili integrando metodi per le misurazioni Traceroute, con filtraggio sul numero di hop e/o di risposte nulle, in modo da ridurre e/o selezionare le misure utili all'analisi. Sarà anche possibile aggiungere un AS path e/o un path per le coordinate, cosi da poter localizzare le aeree geografiche e dunque circoscrivere i probe utilizzati.

2 Architettura

2.1 RIPE Atlas

Per ottenere dati su probe, anchor[1], measurement e result, sono state utilizzate alcune API messe a disposizione da RIPE Atlas[Atl]. L'API permettono richieste sia di tipo GET che di tipo POST, nel nostro caso si utilizza solo la prima tipologia, avendo poi come base l'URL: *https://atlas.ripe.net/api/v2*. È possibile, con questo ultimo, accedere a diversi oggetti, dei quali ce ne interessano in particolar modo due tipi: gli oggetti */probes* e gli oggetti */measurements*, e su ognuno dei quali è possibile utilizzare diversi parametri di ricerca.

2.1.1 Probe

I parametri di ricerca per i probe da poter usare sono molti, di questi sono stati utili al progetto:

- **status_name**: parametro di tipo enumerativo utile a selezionare i probe in base allo stato descritto. Nel nostro caso solo lo stato "connected"
- **radius**: permette di descrivere un'area circolare, settando le coordinate geografiche del centro e il raggio in kilometri, nella quale cercare i probe

[1] Un anchor è un probe con più capacità di misurazione ed è spesso target di misurazioni regionali. È possibile utilizzare l'API anchor_measurements per selezionare misure fatte unicamente verso anchor

- **is_anchor**: parametro di tipo booleano, utile per selezionare solo anchor (true) o anchor e probe (false)
- **country_code**: permette di selezionare i probe per nazione di appartenenza
- **format**: specifica il formato della risposta HTTP; nel nostro caso JSON

L'API darà come risposta una prima pagina contenente una lista di probe che rispettano i criteri settati ed un link ad una pagina successiva, se esiste. Dai risultati ottenuti estraiamo alcuni attributi riguardati i probe:

- **prb_id**[2]: ID intero, che identifica il probe
- **address_v4/address_v6**: ultimo indirizzo IP v4/v6 assegnato al probe

2.1.2 Measurement

Per i parametri dei measurement invece abbiamo ugualmente molte opzioni, di cui quelli utilizzati sono:

- **is_oneoff**: paramentro di tipo booleano, permette di scegliere fra due tipologie di measurement: one-off (true) e periodic (false), se non è specificato allora prende entrambe
- **type**: parametro enumerativo, permette di scegliere quale tipo di misura vogliamo considerare (ping, traceroute ecc.)
- **target_ip**: permette di isolare i measurement in base al target IP
- **status**: parametro enumerativo, con il quale è possibile filtrare in base allo stato del measurement, nel nostro caso saranno *ongoing* o *stopped*

L'API restituirà, in maniera analoga all'API dei probe, una prima pagina con i measurement e, se esiste, il link a una pagina successiva.
Di ogni measurement prendiamo in considerazione i seguenti attributi:

- **id**: identificatore del measurement
- **start_time/ stop_time**: contiene il timestamp UNIX dell'avvio/ della terminazione del measurement
- **target_ip**: indirizzo target del measurement

[2]Solitamente gli anchor hanno ID più alti rispetto ai semplici probe

2.1.3 Result

Ottenuti gli ID dei measurement che ci interessano, possiamo accedere ai risultati utilizzando nuovamente l'API delle misurazioni, cercando nella directory "/results", preceduta dall'ID della misurazione che ci interessa (/id/results). È possibile anche selezionare i risultati che ci interessano utilizzando parametri come per probe e measurement. Quelli che ci interessano sono :

- **start/stop**: consente di stabile un limite inferiore/superiore al timestamp UNIX che vogliamo selezionare
- **probes_id**: consente di definire una lista di probe sorgenti della quale vogliamo trovare i risultati

A differenza di probe e measurement, la risposta per i result sono caricati in unica pagina, il che crea problemi nel caso sia molto grande. Per ovviare a questa problematica il DSL da noi strutturato utilizza tecniche di cacheing.

2.2 DSL con Groovy

Groovy, come già detto, è un linguaggio ad alto livello di astrazione, con sintassi molto facile e chiara, caratteristica che lo rende perfetto per implementare il nostro DSL, è anche possibile integrarlo con Java, il che permette di usarne le librerie.
Di seguito sono descritti i costrutti, implementati da Groovy, utilizzati nel progetto.

2.2.1 Closures

Le closure sono uno dei principali costrutti caratteristici del linguaggio Groovy, si possono definire come un frammento di codice anonimo che può essere assegnato ad una variabile. [Dea10b] Questa tipologia di costrutti può utilizzare variabili interne ed esterne, le prime sono visibili solo alla closure.
Ogni closure può essere invocata utilizzando il metodo *call*, oppure, richiamando il nome della closure come in Fig.1.

```
biggest = { number1, number2 -> number1<number2?number2:number1 }
// We can invoke the call method of the Closure class
result = biggest.call(7, 1)
println result
// We can use the closure reference as if it were a method
result = biggest(3, 5)
println result
// And with optional parenthesis
result = biggest 13, 1
println result
```

Figura 1: Closure [Dea10b]

Uno dei modi per utilizzare le closure è implementare delle strutture di controllo o anche strutture stratificate. Prendendo come esempio il codice riportato nella Fig. 2, è possibile implementarlo in vari modi, ad esempio usando strutture stratificate, in questo caso specifico utilizzando il metodo **email** descritto in Fig. 3. Dal codice vediamo che il metodo prende in ingresso una closure, e crea un oggetto di classe **EmailSpec**, descritta in Fig.4, a cui delegare l'implementazione dei metodi utilizzati. Il metodo poi crea una copia della closure cl utilizzando la funzione **rehydrate**, e mediante la stessa, setta le classi delegate, owner e this del clone che sarà poi assegnato alla varibile **code**. Le tre classi accennate hanno le seguenti funzioni[Apaa]:

- **owner**: è l'oggetto nel quale è definita la closure, che a suo volta può essere una classe o una closure
- **this**: è la classe nella quale è definita la closure
- **delegate**: è un oggetto di "third-party" dove le chiamate o le proprietà possono essere risolte/trovate quando non sono definite nel owner o nel this

Una volta creata la copia è possibile reindirizzare la risoluzione dei metodi e delle variabili. Di default sono definiti in **owner** o in **this**, e si usa delegate solo in caso non siano presenti nei primi, ma è possibile infatti cambiare strategia di risoluzione, come nell'esempio, ed utilizzare la tecnica di definizione "DELEGATE ONLY", la quale permette di usare direttamente la classe **delegate**, in questo caso **EmailSpec**. In **EmailSpec** sono definiti tutti i metodi utilizzati dal codice, in particolare si vede anche la definizione di **body** che a sua volta prende in ingresso una closure e la risolve nello stesso modo di **email**.

```
email {
    from 'dsl-guru@mycompany.com'
    to 'john.doe@waitaminute.com'
    subject 'The pope has resigned!'
    body {
        p 'Really, the pope has resigned!'
    }
}
```

Figura 2: Esempio di codice per utilizzo di una struttura costruita con una strategia builder [Apac]

```
def email(Closure cl) {
    def email = new EmailSpec()
    def code = cl.rehydrate(email, this, this)
    code.resolveStrategy = Closure.DELEGATE_ONLY
    code()
}
```

Figura 3: Implementazione del metodo email [Apac]

```
class EmailSpec {
    void from(String from) { println "From: $from"}
    void to(String... to) { println "To: $to"}
    void subject(String subject) { println "Subject: $subject"}
    void body(Closure body) {
        def bodySpec = new BodySpec()
        def code = body.rehydrate(bodySpec, this, this)
        code.resolveStrategy = Closure.DELEGATE_ONLY
        code()
    }
}
```

Figura 4: Implementazione della EmailSpec [Apac]

2.2.2 Maps

Groovy, a differenza di Java, implementa una interfaccia molto semplice per le liste associative, ovvero le maps[Dea10c], le quali si implementano come negli esempi di Fig.5

```
book = [ fileUnder: "Software Development",
         title: "Groovy for DSL" , author: "Fergal Dearle"]
println book
println book['title']
println book.title
```

Figura 5: Implementazione Maps [Dea10c]

2.2.3 Command Chain

Uno strumento particolare di Groovy sono le command chain[Apab], ovvero, il linguaggio permette di omettere le parentesi e i punti per la chiamata dei metodi che si vogliono utilizzare, in questo modo la scrittura del codice è più intuitiva e semplice

come negli esempi di Fig. 6.
Per implementare una command chain è necessario l'utilizzo di closure, per dichiarare il codice che ogni key-word usata utilizza, e delle maps per concatenare, come nell'esempio di Fig.7

```
// equivalent to: turn(left).then(right)
turn left then right

// equivalent to: take(2.pills).of(chloroquinine).after(6.hours)
take 2.pills of chloroquinine after 6.hours

// equivalent to: paint(wall).with(red, green).and(yellow)
paint wall with red, green and yellow

// with named parameters too
// equivalent to: check(that: margarita).tastes(good)
check that: margarita tastes good

// with closures as parameters
// equivalent to: given({}).when({}).then({})
given { } when { } then { }
```

Figura 6: Utilizzo di una command chain [Apab]

```
show = { println it }
square_root = { Math.sqrt(it) }

def please(action) {
  [the: { what ->
    [of: { n -> action(what(n)) }]
  }]
}

// equivalent to: please(show).the(square_root).of(100)
please show the square_root of 100
// ==> 10.0
```

Figura 7: Implementazione di una command chain [Apab]

3 Implementazione DSL

Abbiamo quindi definito gli strumenti necessari per implementare un DSL con il linguaggio Groovy. Nel nostro caso dovrà permettere all'utente di richiedere dati all'infrastruttura RIPE Atlas, su probe, measurement e i relativi result, attraverso l'utilizzo di API.
Il DSL dovrà prevedere un meccanismo di richiesta in cui deve essere possibile stabilire che tipo di dato si vuole analizzare, quali e quanti target si vogliono considerare e con quali parametri, il tutto dovrà essere poi tradotto in una serie di istanze HTTP verso RIPE Atlas per ottenere le informazioni ed eventualmente elaborarle.

3.1 Builder

Per prima cosa dobbiamo definire un builder come in Fig.2, nella quale sarà possibile inserire chiamate ai metodi necessari a settare i parametri per l'interrogazione verso RIPE Atlas, un esempio del nostro builder è presente in List. 1

```
RIPEAtlas {
    obtain traceroute_list
    from anchors
    to anchors
    limit 3 on measurements
    limit 10 on targets
    limit 10 on sources
    limit 50 on results
    filter hop that ">=" of 3
    source_country "IT"
    target_country "DE"
    verbose on
    cache off
    timeframe "2022-01-01 00:00:00",start
    timeframe "2022-05-30 00:00:00",stop
    execute()
}
```

Listing 1: Esempio builder DSL

Poi, seguendo l'esempio in Fig.3, dobbiamo implementare un metodo che supporti il builder sopra descritto. Nel nostro caso lo abbiamo implementato, come si vede in List.2, utilizzando l'attributo **@DelagatesTo**, la quale permette di segnalare al compilatore che la closure potrebbe non contenere tutte le definizioni di cui ha bisogno, e che sarà necessario risolverle cercando nella classe **request**.

```
def RIPEAtlas(@DelegatesTo(request) Closure cl) {
    def show = new request()
    def definitions = new defs()
    def code = cl.rehydrate(show, definitions, this)
    code.resolveStrategy = Closure.DELEGATE_FIRST
    code()
 }
```

Listing 2: Metodo builder DSL

Vediamo poi che la strategia di risoluzione della closure è di tipo "DELEGATE FIRST", ciò implica che nel caso in cui metodi e variabili non siano definibili utilizzando la classe **request**, il compilatore dovrà utilizzare la classe **defs**, nella quale sono definite stringhe costanti contenenti il loro stesso nome, questo stratagemma permetterà di omettere le virgolette nella chiamata di un metodo e nella scrittura dei suoi parametri, come per esempio nel caso di *limit 50 on results* invece che *limit 50 on "results"*. Ovviamente non è stato possibile inserire tutti i valori plausibili, come per esempio le sigle delle nazioni per i metodi *source_country/target_country*.

```
class defs{
    final String avg_ping = "avg_ping"
    final String traceroute_list = "traceroute_list"
    final String one_off = "one_off"
    final String max = "max"
    final String min = "min"
    final String start = "start"
```

```
    final String stop = "stop"
    final String measurements = "measurements"
    final String probes = "probes"
    final String anchors = "anchors"
    final String results = "results"
    final String hop = "hop"
    final String star = "star"
    final String of = "of"
    final String on = "on"
    final String that = "that"
    final String off = "off"
    final String sources = "sources"
    final String targets = "targets"
    final String any = "any"
    final String ping_list = "ping_list"
    final String measurement_list = "measurement_list"
    final String periodic = "periodic"
}
```

Listing 3: Classe Defs

3.2 Parametri e operazioni implementati

In relazione al builder descritto nella List. 1 , sono stati aggiunti parametri e operazioni di ricerca alla classe request, questi sono descritti nella seguente lista:

- *Operazioni*
  - **traceroute_list**: permette di recuperare una lista di risultati dai traceroute measurement, prendendo in considerazione source e target filtrati secondo i parametri di ricerca impostati. I risultati sono poi salvati in un file
- *Parametri*
  - **limit ... on *results***: il metodo è lo stesso utilizzato per probe, anchor, measurement, ma permette di imporre un limite ai result prelevati da ogni measurement. Il funzionamento è basato sul meccanismo command chain.
  - **asn on/ off**: è un flag utilizzato per aggiungere all'IP-path un AS-path nel caso si stia usando l'operazione traceroute_list (con le altre operazioni non ha alcun effetto)
  - **coordinate on/ off**: è un flag utilizzato per aggiungere ad un IP-path un path di coordinate geografiche nel caso si stia usando l'operazione traceroute_list (con le altre operazioni non ha alcun effetto), il funzionamento sarà precisato nel paragrafo 3.3.2
  - **filter ... that ... of ...**: metodo che utilizza il meccanismo di command chain per filtrare i result di una traceroute_list in base a hop e/ o star

All'interno della closure i parametri possono avere un qualsiasi ordine eccetto il metodo **execute**, definito nel codice seguente, poiché esso deve trovarsi necessariamente alla fine in quanto, tutti i parametri necessari alla ricerca devono essere settati prima dell'esecuzione.

```
 def execute = {
    if (reqF.equals("avg_ping")){
        Double result
        if (debug) println("targets")
        getIps(destProbesIps, "dest")
        if (debug) println("sources")
        getIps(sourceProbesIps, "source")
        getMeas(measIds, destProbesIps, "ping")
        result = getAvgPing(measIds, sourceProbesIps)
        println("fine: "+result)
    }
    else if (reqF.equals("ping_list")){
        if (debug) println("targets")
        getIps(destProbesIps, "dest")
        if (debug) println("sources")
        getIps(sourceProbesIps, "source")
        getMeas(measIds, destProbesIps, "ping")
        container c = new container()
        getPingList(measIds, sourceProbesIps, c)
        FileWriter f = new FileWriter("resultsPing_list.txt", true) // semplice salvataggio dei risultati in file diversi o aggiungere separatori per le varie misurazioni e tipologie
        String towrite = JsonOutput.toJson(c.data)
        f.write(towrite)
        f.close()
    }
    else if (reqF.equals("measurement_list")){
        if (debug) println("targets")
        getIps(destProbesIps, "dest")
        if (debug) println("sources")
        getIps(sourceProbesIps, "source")
        getMeas(measIds, destProbesIps, null)
        FileWriter f = new FileWriter("resultsmeasuremnt_list.txt", true)
        String towrite = JsonOutput.toJson(measIds)
        f.write(towrite)
        f.close()
    } else if (reqF.equals("traceroute_list")){
        if (debug) println("targets")
        getIps(destProbesIps, "dest")
        if (debug) println("sources")
        getIps(sourceProbesIps, "source")
        getMeas(measIds, destProbesIps, "traceroute")
        containerT c = new containerT()
        getTracerouteList(measIds, sourceProbesIps, c)
        FileWriter f = new FileWriter("resultsTraceroute_list.txt", true)
        def generator = new JsonGenerator.Options().excludeNulls().build()
        String towrite = generator.toJson(c.data)
        f.write(towrite)
        f.close()
    }
}
```

Listing 4: Codice Execute

3.3 Metodo Traceroute

L'operazione di **traceroute_list** richiama il metodo **getTracerouteList**, implementato nella List.6, che esegue in ordine :

1. Un'operazione di recupero degli indirizzi IP, sia dei target che dei source, attraverso il metodo **getIps** descritto in List. 4

2. Un'operazione di recupero dei measurement, che hanno come target gli indirizzi trovati, usando il metodo **getMeas** implementato in List 5

3. Un'operazione di filtraggio sui measurement in base ai source probe trovati

4. Operazioni opzionali sono quelle di filtraggio su hop e/ o star e/ o di aggiunta tipologie di path, che saranno descritte meglio nei paragrafi 3.3.1 e 3.3.2 .

```
void getIps(LinkedList<?> p, String s){//recupera indirizzi e id delle
    probes che rispettano le specifiche
    if (debug) println("ips")
    if (s.equals("dest") && targetIps != null) return
    String res
    StringBuilder bres = new StringBuilder()
    def jsonSlurper = new JsonSlurper()
    String url2 = "https://atlas.ripe.net/api/v2/probes/?format=json&
    status_name=Connected"
    if (s.equals("source")){
        if ((sLat != 0 || sLon != 0) && sRad != 0)
            url2+= "&radius="+sLat+","+sLon+":"+sRad
        else if (sCountry != null)
            url2+="&country_code="+sCountry // il controllo sulle nazioni
         if (fromWhat.equals("anchors"))     // e' gia presente
            url2+="&is_anchor=true"

    }
    else if (s.equals("dest")){
        if ((dLat != 0 || dLon != 0) && dRad != 0)
            url2+= "&radius="+dLat+","+dLon+":"+dRad
        else if (dCountry != null)
            url2+="&country_code="+dCountry
        if (toWhat.equals("anchors"))
            url2+="&is_anchor=true"
    }
    if (debug) println(url2)
    res = sendRequestWithCacheCheck(url2,bres)
    //println("sono in getIps"+res)
    if (res.equals("errore") || res.equals("")){
        return
    }
    def object = jsonSlurper.parseText(res)
    if (object.count != 0) {
        int count = 0
        int limit = object.count
        if (s.equals("source")) {
            if(source_limit!=0)limit = source_limit
        }
        else {
            if(target_limit!=0)limit = target_limit
        }
        while (count < limit){
            for (int i = 0; i< object.results.size() && count < limit; i++)
            {
                if (!object.results[i].address_v4.equals(null)) {
                    if (s.equals("source")){
                        if (addr.equals("IPv4")){
                            if (debug)
```

```
                                    println(s+" "+object.results[i].address_v4+", id "+object.results[i].id)
                                sourceData z = new sourceData(object.results[i].id,object.results[i].address_v4)
                                p.add(z)
                            }
                            else {
                                if (debug)
                                    println(s+" "+object.results[i].address_v6+", id "+object.results[i].id)
                                sourceData z = new sourceData(object.results[i].id,object.results[i].address_v6)
                                p.add(z)
                            }

                            count++
                        }
                        else {
                            if (addr.equals("IPv4")){
                                if (debug)
                                    println(s+" "+object.results[i].address_v4+", id "+object.results[i].id)
                                p.add(object.results[i].address_v4)
                            }
                            else{
                                if (debug)
                                    println(s+" "+object.results[i].address_v6+", id "+object.results[i].id)
                                p.add(object.results[i].address_v6)
                            }
                            count++
                        }
                    }
                }
                if (debug) println("count = "+count)
                if (count >= limit) break
                if (object.next != null){ //vado alla prossima pagina di risultati
                    url2 = object.next
                    res = sendRequestWithCacheCheck(url2,bres)
                    object = jsonSlurper.parseText(res)
                }
                else {
                    if (debug && (target_limit!=0 || source_limit!=0))
                        println("numero risultati trovati per "+s+" minore del limite imposto")
                    break
                }
            }
        } //get ips
        else {
            if (debug) println("nessun risultato trovato per "+s)
        }
}
```

Listing 5: getIps

```

void getMeas(LinkedList<?> meas, LinkedList<?> ips, String type)
{// recupera i measurements che hanno come target membri della lista ips
    if (ips.isEmpty()) {
        if (debug) println("Non ci sono ip di probes target")
        return
    }
```

```
    if (debug) println("start of measurements")
    String res
    StringBuilder bres = new StringBuilder()
    ListIterator<String> ipIter
    Integer count_tot = 0
    if (targetIps != null){ //do la precedenza ai targetIps se sono stati settati dall'utente
        ipIter = targetIps.listIterator()
    }
    else ipIter = ips.listIterator()
    def jsonSlurper = new JsonSlurper()
    for (int i = 0; ipIter.hasNext(); i++ ){
        bres = new StringBuilder()
        String url2 = "https://atlas.ripe.net/api/v2/measurements/?format=json&target_ip="+ipIter.next()
        if (oneOff.equals("one_off")) url2+="&is_oneoff=true"
        else if (oneOff.equals("periodic")) {
            url2+="&is_oneoff=false"
        }
        if (type != null) url2+="&type="+type

        if (time_stop !=0) {
            url2+="&start_time__lt="+time_stop // deve controllare anche i measurement che sono partiti prima di time_start
            url2+="&status=Ongoing,Stopped"
        }
        //if (time_stop != 0) url2+="&stop_time__lte="+time_stop+"&stop_time__gte="+time_start // ma lo stop time deve essere comunque nell' interevallo
        res = sendRequestWithCacheCheck(url2,bres)
        println("sono in getMeas: "+url2)
        if (res.equals("errore") || res.equals("")){
            continue
        }
        def object = jsonSlurper.parseText(res)
        if(debug)println("numero di Meas trovati in totale: "+object.count)
        if(object.count != 0){
            int count = 0
            int limit = object.count
            if(meas_limit!=0) {limit = meas_limit}
            while (count < limit){
             for(int j = 0; j < object.results.size() && count < limit;j++)
             {
              if(object.results[j].status.name == "Stopped" && time_start!=0 && time_stop!=0)
               {

                if(object.results[j].stop_time >= time_start || object.results[j].status.when >= time_start)
                { // l'importante e' che il mesaurement si fermi dopo il time start
                    measItems m = new measItems(object.results[j].id,
            object.results[j].start_time,object.results[j].stop_time,
            object.results[j].target_ip,object.results[j].status.name)
                    if (debug)
                        println("adding id: "+object.results[j].id+" -- start_time: "+object.results[j].start_time+" -- stop_time: "+object.results[j].stop_time+" -- status: "+object.results[j].status.name)
                    meas.add(m)
                    count++
                    count_tot++
                    if (debug) println("numMeas: "+(count))
                }else{
                    if (debug)
```

```
                        println("not adding id: "+object.results[j].id+" -- start_time: "+object.results[j].start_time+" -- stop_time: "+object.results[j].stop_time+" -- status: "+object.results[j].status.name)
                }
            }
                else {
                    measItems m = new measItems(object.results[j].id,object.results[j].start_time,object.results[j].stop_time,object.results[j].target_ip,object.results[j].status.name)

                    if (debug)
                        println("adding id: "+object.results[j].id+" -- start_time: "+object.results[j].start_time+" -- stop_time: "+object.results[j].stop_time+" -- status: "+object.results[j].status.name)
                    meas.add(m)
                    count++
                    count_tot++
                    if (debug) println("numMeas: "+(count))
                }
            }

            if (count >= limit) break
            if (object.next != null){ //vado alla prossima pagina di risultati
                bres = new StringBuilder()
                url2 = object.next
                res = sendRequestWithCacheCheck(url2,bres)
                object = jsonSlurper.parseText(res)
            }
            else {
            if (debug && meas\_limit != 0)
                    println("numero di measurements trovati    inferiore al limite imposto")
                break
            }
      }
        }
    }//get a measurement for each ip
    if(debug) println("Measurement totali: "+count_tot)
    }
```

Listing 6: getMeas

```
void getTracerouteList(LinkedList<?> p, LinkedList<?> s, containerT c){ // agigustare il limit result
    if (p.isEmpty()){
        if (debug) println("Non ci sono measurements")
        return
    }
    if (s.isEmpty()){
        if (debug) println("Non ci sono ip di probe source")
        return
    }
    if (debug) println("start of results")
    StringBuilder bres
    String res
    ListIterator<measItems> measIt = p.listIterator()
    ListIterator<sourceData> sipsIt = s.listIterator()

    def jsonSlurper = new JsonSlurper()
    for (int i = 0; i < p.size(); i++){
        bres = new StringBuilder()
        measItems m = measIt.next()
```

```
        String url2 = "https://atlas.ripe.net/api/v2/measurements/"+m.id+"/results/?format=json&probe_ids="
        sipsIt = s.listIterator()
        while (sipsIt.hasNext()){
            sourceData tmp = sipsIt.next()
            url2+=tmp.id
            if (sipsIt.hasNext()) url2+=","
        }
        if (time_start != 0 && time_start > m.start_time)
            url2 = url2 + "&start=" + time_start
        else
            url2 = url2 + "&start=" + m.start_time
         if (time_stop != 0 && time_stop > m.start_time)
            url2 = url2 + "&stop=" + time_stop
        else
            url2 = url2 + "&stop=" + (m.start_time+1)
        if (debug) println(url2)
        res = sendRequestWithCacheCheck(url2,bres)

        if (res.equals("errore") || res.equals("")){
            continue
        }
        def object = jsonSlurper.parseText(res)
        if (debug) println("Size: "+object.size())
         if(object.size() != 0){
            int count = 0
            int limit = object.size()
            if (res_limit!=0){
               if(limit>res_limit)
                    limit = res_limit
            }

            for (int j = 0; j < object.size() && count<limit; j++){
                while (sipsIt.hasNext()){
                    sourceData tmp = sipsIt.next()
                    int count_star = 0
                    int asn
                    Double lat
                    Double lon
                    boolean star = false
                    List<hop> lista_hop = new LinkedList<>()


                    if(hop_limit>=0 &&
                    !comparazione(segno_filtro,
                    object[j].result.size(),hop_limit))
                    {
                        if(debug)
                           println("Il numero di hop non rispetta il filtraggio richiesto :"+object[j].result.size())
                        break
                    }

                    for(int k = 0;k<object[j].result.size();k++){
                        List<responseTrace> risposte = new LinkedList<>()
                        star = false
                        for(int l = 0;l<3;l++){
                          if(object[j].result[k].result[l].x != null){
                           risposte.add(new responseTrace(
                           object[j].result[k].result[l].x))
                           if(star_limit>=0 || asn_on || coordinate_on)
                                star = true
                        } else
                        {
```

```
                            risposte.add( new responseTrace(
                            object[j].result[k].result[l].from,
                            bject[j].result[k].result[l].ttl,
                            object[j].result[k].result[l].size,
                            object[j].result[k].result[l].rtt))
                        }
                    }
                        if(star_limit>=0 && star == true){
                            count_star++
                        }

                       if(!star && asn_on){
                            StringBuilder bres2 = new StringBuilder()
                            String urlAS = "https://stat.ripe.net/data/prefix-overview/data.json?resource="+object[j].result[k].result[0].from
                            String res_as = sendRequestWithCacheCheck(urlAS,bres2)
                            if(res_as.equals("errore")||res_as.equals(""))
                               asn = -2
                            else{
                                def asn_obj = jsonSlurper.parseText(res_as)
                                String str_tmp = asn_obj.data.asns.asn
                                if(str_tmp.equals("[]")){
                                    asn = -1
                                } else{
                                    str_tmp = str_tmp.replace('[','')
                                    str_tmp = str_tmp.replace(']','')
                                    asn = Integer.parseInt(str_tmp)
                                }
                            }

                        }

                        if(!star && coordinate_on){
                            StringBuilder bres2 = new StringBuilder()
                            String urlCOOR ="https://stat.ripe.net/data/maxmind-geo-lite/data.json?resource="+object[j].result[k].result[0].from
                            String res_coor = sendRequestWithCacheCheck(urlCOOR,bres2)
                            if(res_coor.equals("errore")||res_coor.equals("")){
                                lat = Double.NaN
                                lon = Double.NaN
                            }
                            else{
                                def coor_obj = jsonSlurper.parseText(res_coor)
                                String str_tmp = coor_obj.data.located_resources.locations.longitude
                                if(str_tmp.equals("[]")){
                                    lon = Double.NaN
                                }else{
                                    str_tmp = str_tmp.replace('[','')
                                    str_tmp = str_tmp.replace('[','')
                                    str_tmp = str_tmp.replace(']','')
                                    str_tmp = str_tmp.replace(']','')
                                    lon = Double.parseDouble(str_tmp)
                                }

                                str_tmp = coor_obj.data.located_resources.locations.latitude
                                if(str_tmp.equals("[]")){
                                    lon = Double.NaN
```

```
                                }else{
                                    str_tmp = str_tmp.replace('[','')
                                    str_tmp = str_tmp.replace('[','')
                                    str_tmp = str_tmp.replace(']','')
                                    str_tmp = str_tmp.replace(']','')
                                    lat = Double.parseDouble(str_tmp)
                                }
                            }

                        }



                        if(coordinate_on && asn_on
                        && !Double.isNaN(lat) && !Double.isNaN(lon)
                        && asn != -2)
                            lista_hop.add(new hop(object[j].result[k].hop,
                            risposte,asn,lon,lat))
                        else if(coordinate_on && !asn_on
                             && !Double.isNaN(lat) && !Double.isNaN(lon))
                            lista_hop.add(new hop(object[j].result[k].hop,
                            risposte,lon,lat))
                        else if(!coordinate_on && asn_on && asn != -2)
                            lista_hop.add(new hop(object[j].result[k].hop,
                            risposte,asn))
                        else
                            lista_hop.add(new hop(object[j].result[k].hop,
                            risposte))

                    }

                    if(!comparazione(segno_filtro,count_star,star_limit) &&
    star_limit>=0){
                        if(debug)println("Il numero di star non rispetta il
    filtraggio richiesto :"+count_star)
                        break
                    }

                    count++
                    Date date = new Date((long)object[j].timestamp*1000)
                    Calendar calendar = Calendar.getInstance()
                    calendar.setTime(date)
                    int mese = calendar.get(calendar.MONTH) + 1
                    c.data.add((new traceroutelist(""+
                    calendar.get(calendar.DAY_OF_MONTH)+
                    "/"+mese+"/"+calendar.get(calendar.YEAR),
                    object[j].src_addr, object[j].dst_addr,lista_hop)))
                    if (debug) println("id of measurement: "+m.id)
                    break
                }
                    sipsIt = s.listIterator()

                    if(count>=limit)break
                }

        }
    }
}
```

Listing 7: getTraceroutList

Il risultati di ogni richiesta di tipo traceroute sono incapsulati in base al genere di risultato e a quali e quanti tipi di path che si vogliono. Questa possibilità è

data dall'incapsulamento di più classi e la gestioni di più costruttori specializzati, implementati nella listing sottostante.

```
class containerT{
    List<tracerouteList> data = new LinkedList<>()
}

class responseTrace{
    String ipresponse
    Double ttl
    Integer size
    Double rtt
    String x
   responseTrace (String ip, Double t, Integer s, Double r){
        ipresponse = ip
        ttl = t
        size = s
        rtt = r
    }

    responseTrace (String e){
        x = e
    }

}

class hop {
    Integer hop
    Integer asn
    Double lon
    Double lat
    List<responseTrace> risposte

    hop (Integer h, List<responseTrace> l,Integer a,Double lo,Double la){
        hop = h
        risposte = l
        asn = a
        lon = lo
        lat = la
    }

    hop (Integer h, List<responseTrace> l,Integer a){
        hop = h
        risposte = l
        asn = a
    }
    hop (Integer h, List<responseTrace> l,Double lo,Double la){
        hop = h
        risposte = l
        lon = lo
        lat = la
    }
    hop (Integer h, List<responseTrace> l){
        hop = h
        risposte = l
    }
}
class tracerouteList{ // struttura per il salvataggio di una lista di ping;
     potremmo aggiungere altre misurazioni oltre a quella media e altre
    informazioni sul ping

    String data
    String ipsource
    String iptarget
```

```
    List<hop> list_route
    traceroutelist(String d,String ips,String ipt,List<hop> l)
    {
       data = d
       ipsource = ips
       iptarget = ipt
       list_route = l
    }
}
```

Listing 8: Struttura dati

3.3.1 Filtri

Come abbiamo detto è possibile applicare dei filtri di selezione mediante l'uso del metodo **filter ... that ... of ...** , implementato come in List.7: il primo parametro riguarda l'argomento su cui si vuole filtrare, nel nostro caso o **hop** o **star**, il secondo permette di scegliere quale operatore di comparazione si vuole usare (es $>$,$>=$,$! =$ ecc.), il terzo è la quantità con il quale si vuole comparare. Un esempio di utilizzo si trova alla riga 9 del List.1 (filter hop that "" of 3), che si traduce come *seleziona solo i risultati che hanno un numero di hop maggiore o uguale a 3.* Il confronto viene effettuato chiamando il metodo **comparazione**.

```
     def filter(n) {
        [that: {String sign->[of:{Integer v ->
            switch (n){
               case "hop":
                    hop_limit = v
                    break
                case "star":
                    star_limit = v
                    break
            }

            segno_filtro = sign
            }]
        }]
    }
```

Listing 9: command chain Filter

```
     boolean comparazione(String segno,Integer valore1,Integer valore2){
        switch(segno){
            case "<":
                return valore1<valore2?true:false
                break
            case "<=":
                return valore1<=valore2?true:false
                break
            case ">":
                return valore1>valore2?true:false
                break
            case ">=":
                return valore1>=valore2?true:false
                break
            case "=": case "":case " ":
                return valore1==valore2?true:false
                break
            case "!=":
```

```
                return valore1!=valore2?true:false
                break
            default :
                {
                println("Segno non valido")
                return true
                }

        }
    }
```

Listing 10: Metodo di confronto

Hop: L'argomento **hop** permette di filtrare i risultati di tipo traceroute in base al numero di hop eseguiti, il codice che effettua il filtraggio è implementato in List.9, come si vede è un semplice costrutto di selezione; **hop_limit** viene settato alla riga 5 del List. 7.

```
1 ...
2 if(hop_limit>=0 && !comparazione(segno_filtro,object[j].result.size(),
    hop_limit)){
3     if(debug) println("Il numero di hop non rispetta il filtraggio
    richiesto :"+object[j].result.size())
4     break
5 }
6   ...
```

Listing 11: Seleziona per hop

Star: L'argomento **star** permette di filtrare i risultati di tipo traceroute in base al numero di risposte mancanti, le quali sono indicate con "***" nei result delle tracerout_list. Il filtro è implementato come il precedente, e la variabile **count_star** è incrementata all'interno delle righe 76-78 del List.6. **star_limit** viene settato alla riga 8 del List. 7.

```
1 ...
2 if(!comparazione(segno_filtro,count_star,star_limit) && star_limit>=0){
3     if(debug)println("Il numero di star non rispetta il filtraggio
    richiesto :"+count_star)
4     break
5     }  ...
```

Listing 12: Seleziona per star

3.3.2 API per ASN e coordinate

Oltre al filtraggio, come abbiamo visto nel paragrafo precedente, è possibile anche aggiungere 2 tipi di path: uno formato dall'ASN[3], il quale si attiva settando la variabile booleana **asn_on** su true ed è gestito dalla closure implementata nella List.11; il secondo path invece è composto dalle coordinate di ogni probe che ha risposto positivamente alla richieste traceroute, attivato tramite una variabile booleana (**coordinate_on**) gestita come la precendete.

[3]Autonomus System Number

```
    def asn = { String q ->
        if (q.equals("on")) asn_on = true
        if (q.equals("off")) asn_on = false
    }

    def coordinate = { String q ->
        if (q.equals("on")) coordinate_on = true
        if (q.equals("off")) coordinate_on = false
    }
```

Listing 13: Asn e Coordinate

Per ottenere le informazioni sull'ASN e sulle coordinate dei probe, si sono utilizzate 2 API che appartengono all'infrastruttura RIPEStat; entrambe hanno come base l'URL *https://stat.ripe.net/data/*, dalla quale è possibile arrivare all'API utile. La prima, utilizzata per richiedere ASN, è **prefix-overview**, la quale richiede in ingresso il formato con cui si vuole il risultato, nel nostro caso JSON, e l'indirizzo IP del probe di cui si vuole l'ASN; viene poi fatto il parse del risultato e aggiunto come attributo al result della traceroute_list.
La seconda, utilizzata per le coordinate del probe, è **maxmind-geo-lite**, come la precedente richiede in ingresso il formato con cui si vuole l'output e l'indirizzo IP della risorsa interessata, per poi fare il parse e aggiungere come attributo i risultati. Le risposte di entrambe l'API possono essere negative, o meglio, per l'ASN è plausibile che non sia possibile rilevare l'ASN per cui come responso nella traceroute_list ci sarà un -1, mentre per le coordinate anche queste potrebbero essere non rilevabili, per cui l'attributo sarà vuoto.

4 Modifiche

4.1 Modifica dell'intervallo di tempo per i measurement

Oltre allo sviluppo di nuove operazioni da poter utilizzare con il DSL, sono state apportate anche alcune modifiche, nello specifico nel metodo **getMeas**. Queste riguardano i parametri per impostare un range di tempo, ovvero: è stata modificata la parte di codice riguardante l'intervallo di tempo dei measurment, o meglio, stabilito un lasso di tempo, con A come data iniziale e B data finale, si vogliono prelevare i risultati dei vari measurement in questo spazio di tempo, bisogna però tener conto che alcune misurazioni possono avere una data di inizio precedente alla data A, aver generato risultati all'interno dell'intervallo e non essere ancora terminate, per cui limitare la selezione utilizzando solo i parametri **start_time** e **stop_time** (e i loro derivati) porterebbe a non considerare una quantità di misurazioni non indifferente, poiché si tralascerebbero molti measurement che sono ancora attivi o che hanno l'attributo **start_time** precedente ad A. Per ovviare al problema si è deciso di aggiungere il parametro **status**, di tipo enumerativo, che esprime lo stato della misurazione, i valori che ci interessano sono due : *Ongoing*, che permette di considerare anche misurazioni non terminate, ma se usato da solo, non permette di considerare l'attributo **stop_time** di un measurement poiché sarebbe null, ed

avrebbe anche priorità maggiore rispetto a quest'ultimo, per cui consideriamo anche il valore *Stopped*, che consente di esaminare anche misurazioni terminate, senza includere il parametro **stop_time**,il quale verrà poi analizzato, in un costrutto di selezione, una volta ottenuti i risultati dell'API, dal quale si estrapolerà la lista effettiva dei measurement interessati .

5 Esempio di Utilizzo

Uno dei molti esempi di utilizzo che si possono fare usufruendo delle funzioni, appartenenti al DSL descritte in precedenza, è il tracciamento e la visualizzazione su mappa dei percorsi che i pacchetti seguono attraverso i vari probe delle diverse nazioni, come nell'esempio di builder implementato di seguito.

```
1     RIPEAtlas {
2     obtain traceroute_list
3     from anchors
4     to anchors
5     limit 1 on measurements
6     limit 5 on targets
7     limit 5 on sources
8     limit 20 on results
9     filter star that "<=" of 0
10    coordinate on
11    asn on
12    source_country "IT"
13    target_country "DE"
14    verbose on
15    cache off
16    timeframe "2023-01-01 00:00:00",start
17    timeframe "2023-08-27 00:00:00",stop
18
19    execute()
20 }
```

Listing 14: Esempio di utilizzo

Nello specifico questo builder permette di ottenere misurazioni che: sono state fatte tra il 01/01/2023 e il 27/08/2023, hanno origine da 5 anchor italiani e terminano in 5 anchor tedeschi, con un limite di 1 measurement per ogni probe e di 20 result per ogni measurement, quindi un massimo di 100 result. Ogni result viene poi controllato in modo tale che non vi siano risposte nulle (riga 9) e si attivano sia il path di coordinate che di ASN, purtroppo il filtraggio non permette di scremare i risultati di cui non si possono avere le coordinate o l'ASN, quindi alcuni probe potrebbero non essere disegnati nella mappa.
Dalla lista dei risultati è possibile ottenere, attraverso conversioni e aggiustamenti, una tabella in formato CSV, con la formattazione visibile in Fig.8, successivamente mediante l'uso di software per l'analisi e stampa di dati, nel nostro caso è stato utilizzato il software gratuito QGIS v. 3.32.2 , è possibile visualizzare una mappa come in Fig.9, dove è raffigurato il percorso di un solo result. Alcune etichette non sono visibili poiché molti dispositivi hanno coordinate molto vicine, per cui non è possibile visualizzarli con il fattore di scala dell'immagine. Nel esempio si vede come l'hop numero 5 (etichetta rossa), sia fuori dall'Europa e dalle Fig.4 e 5 vediamo le

sue misurazioni. Con QGIS è possibile anche filtrare i dati in vari modi, ad esempio potrebbe essere utile filtrare per rtt oppure per numero di hop, difatti grazie al software è possibile ottenere tabelle elaborate in modo da poter visualizzare meglio i tracciati e le misure, di conseguenza è possibile individuare hop con una risposta lenta, oppure, individuare probe che appartengono ad uno stesso AS.

ID	ipsource	iptarget	hop	asn	lon	lat	ipresponse	ttl	size	rtt	rtt2	rtt3
1	213.212.129.68	194.150.191.46	1	12779	12.1097	43.1479	213.212.129.65	255	28	0.599	0.621	0.688
1			2	12779	7.6639	45.0914	213.254.30.220	254	68	1.049	0.97	0.85
1			3	12779	7.6639	45.0914	213.254.31.239	253	68	11.98	11.426	11.377
1			4	-1	9.491	51.2993	80.81.192.187	252	28	11.866	11.615	11.566
1			5	201011	9.9532	53.5582	80.255.15.250	250	28	15.154	15.068	14.623
1			6	201011	9.9532	53.5582	80.255.15.14	247	28	14.344	14.241	14.126
1			7	62313	11.034	49.415	194.150.190.35	247	28	14.111	14.131	14.185
1			8	62313	11.034	49.415	194.150.191.46	55	48	14.423	14.549	14.378
2	217.29.76.27	194.150.191.46	1	16004	12.1097	43.1479	217.29.76.253	255	28	0.683	0.989	0.926
2			2	41327	9.1922	45.4722	185.157.229.56	254	28	2.95	0.709	0.791
2			3	41327	12.4278	37.7964	185.157.228.21	249	28	10.117	10.01	10.318
2			4	-1	9.491	51.2993	80.81.192.187	249	28	10.48	10.44	10.523
2			5	201011	9.9532	53.5582	80.255.15.250	249	28	13.719	13.818	13.73
2			6	201011	9.9532	53.5582	80.255.15.14	247	28	13.349	12.98	12.782
2			7	62313	11.034	49.415	194.150.190.35	247	28	12.673	13.072	12.66
2			8	62313	11.034	49.415	194.150.191.46	55	48	13.031	13.08	12.917
3	213.212.129.68	194.150.191.46	1	12779	12.1097	43.1479	213.212.129.65	255	28	0.746	0.643	0.726
3			2	12779	7.6639	45.0914	213.254.30.220	254	68	0.951	0.855	0.914
3			3	12779	7.6639	45.0914	213.254.31.239	253	68	12.952	11.368	11.511
3			4	-1	9.491	51.2993	80.81.192.187	252	28	11.632	11.511	11.546
3			5	201011	9.9532	53.5582	80.255.15.250	250	28	14.491	14.637	14.372
3			6	201011	9.9532	53.5582	80.255.15.14	247	28	14.318	14.121	14.284
3			7	62313	11.034	49.415	194.150.190.35	247	28	14.074	14.217	14.055
3			8	62313	11.034	49.415	194.150.191.46	55	48	14.242	14.315	14.328

Figura 8: Tabella dati

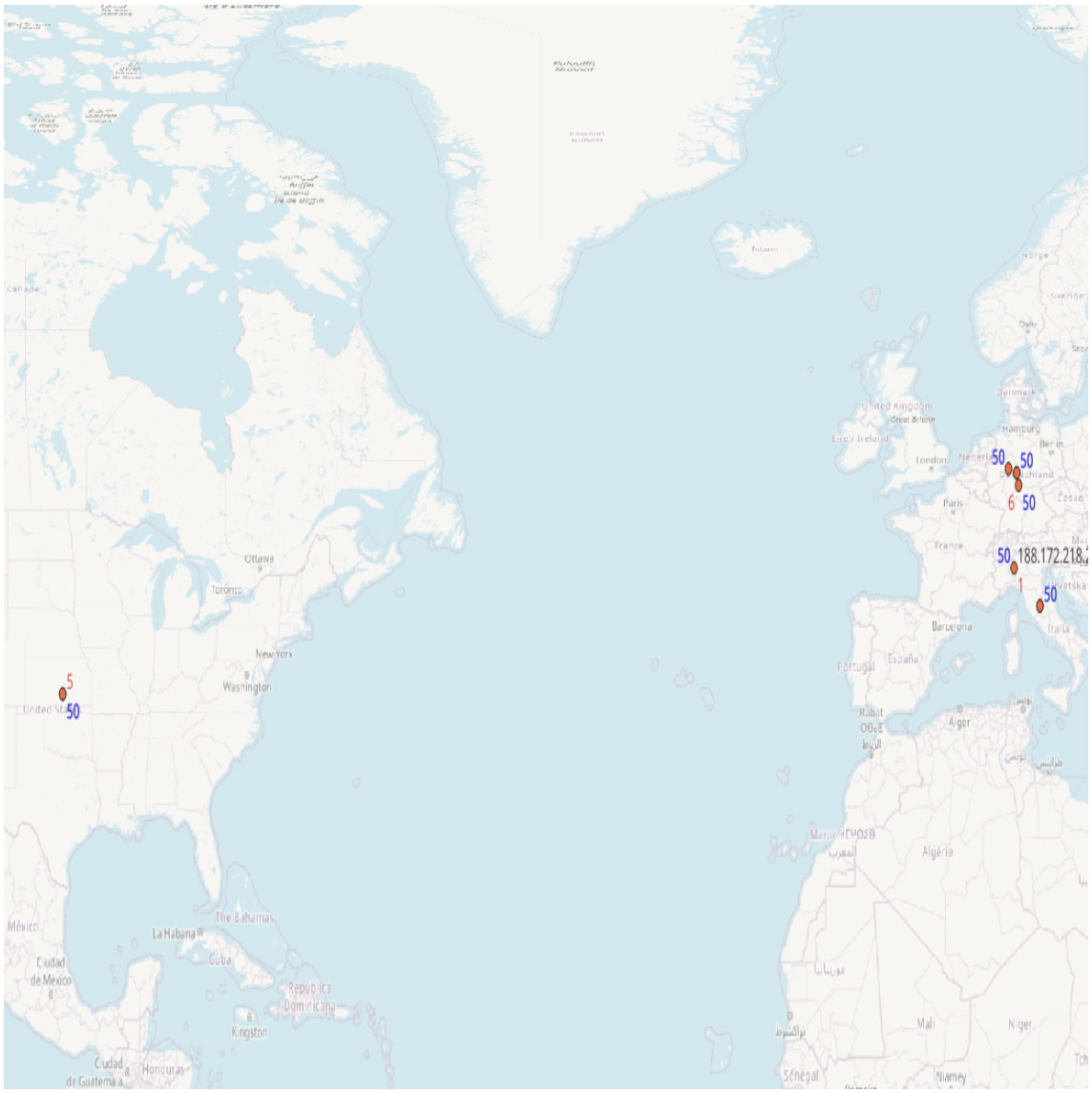

Figura 9: Mappa probe

DataTraceroute3 — Elementi Totali: 9, Filtrati: 9, Selezionati: 0

	ID	ipsource	iptarget	hop	asn	lon	lat
1	50	188.172.218.22	134.169.3.165	1	42473	9,1922	45,4722
2	50	NULL	NULL	2	6762	12,1097	43,1479
3	50	NULL	NULL	3	6762	12,1097	43,1479
4	50	NULL	NULL	4	3356	-97,822	37,751
5	50	NULL	NULL	5	3356	-97,822	37,751
6	50	NULL	NULL	6	3356	9,6975	50,5338
7	50	NULL	NULL	7	680	9,491	51,2993
8	50	NULL	NULL	8	680	8,5628	51,5525
9	50	NULL	NULL	9	680	9,491	51,2993

Mostra Tutti gli Elementi

Figura 10: Dati parte 1

ipresponse	ttl	size	rtt	rtt2	rtt3
188.172.218.21	64	76	0,297	0,323	0,41
195.22.205.24	254	28	0,746	0,517	0,505
195.22.196.71	252	28	1,488	1,02	1,047
4.68.73.37	251	28	1,135	1,066	2,058
4.69.167.118	250	28	23,258	14,504	16,297
62.67.67.154	250	68	21,532	21,558	21,66
188.1.144.134	247	68	34,139	35,38	34,085
188.1.236.190	246	28	23,174	23,145	23,051
134.169.3.165	54	48	27,159	27,146	27,239

Figura 11: Dati parte 2

6 Conclusioni

Abbiamo quindi visto come implementare un DSL mediante l'utilizzo di altri linguaggi di alto livello di astrazioni, con costrutti e funzionalità particolari, che hanno permesso l'implementazione del DSL per le misure di Internet, integrandole con API di strutture come RIPE Atlas si potrebbe quindi pensare di espandere il dominio ad altre strutture simili che potrebbero fornire altri tipi di dato, o comunque estendere ad altre misurazioni ,oltre che di tipo ping e traceroute (come dns,http ecc) o anche implementare nuovi filtri cosi da avere risultati sempre più precisi. In conclusione, come abbiamo visto dall'esempio fatto in precedenza, un Domain-Specific Language può essere uno strumento estremamente utile, poiché con poche righe di codice possiamo ottenere una quantità di dati enorme, oltretutto con un sintassi molto semplice e leggibile, usufruibile sia da utenti con poca o senza esperienza nella programmazione sia da utenti esperti.

Bibliografia

[Dea10a] Fergal Dearle. "Groovy for Domain-Specific Languages". In: Packy Publishing Ltf, 2010. Cap. DSL: New name for an old idea.

[Dea10b] Fergal Dearle. "Groovy for Domain-Specific Languages". In: Packy Publishing Ltf, 2010. Cap. Closures.

[Dea10c] Fergal Dearle. "Groovy for Domain-Specific Languages". In: Packy Publishing Ltf, 2010. Cap. Native support for list and maps.

[Apaa] Apache. *Apache groovy closures owener, this and delegate documentation.* URL: `https://groovy-lang.org/closures.html#closure-owner`.

[Apab] Apache. *Apache groovy command chain documentation.* URL: `https://groovy-lang.org/dsls.html#_command_chains`.

[Apac] Apache. *Apache groovy DelagateTo and example of builder.* URL: `https://groovy-lang.org/dsls.html#section-delegatesto`.

[Atl] RiPE Atlas. *Ripe atlas rest api documantation.* URL: `https://atlas.ripe.net/docs/apis/rest-api-reference/`.